



UNIT 2: DATA TYPES

Prachi Surlakar
Assistant Professor
Computer Engineering Department
Goa College of Engineering

Unit - 2

Introduction: Features of Python, execution of Python programs, Python virtual machines, memory management, garbage collection, comparison between C and Python.

Data Types: Comments, docstrings, built-in data types, strings, sets, literals, user-defined data types, constants, identifiers, reserved words and naming conventions in python.

Operators: Arithmetic, assignment, unary, relational, logical, Boolean, bitwise, membership, identity operators, operator precedence and associativity.

Control statements: if, if-else, if-elif else, while, for, nested loops, break, continue, pass, assert and return statements

12

2. Dr. R. Nageswara Rao; "Core Python Programming", Dreamtech press, Third edition, 2018.

Simple python program

```
# First Python program to add two numbers  
a = 10  
b = 15  
c = a + b  
print("Sum= ", c)
```

Output:

```
Sum= 25
```

COMMENTS IN PYTHON

Comments are **non executable statements**. It means neither python compiler nor PVM will execute them.

There are two types of comments in python

1. Single line comments 2. Multiline comments

1. Single line comments

- These comments start with a hash(#) symbol
- Entire line till the end must be treated as comment.

```
# To find sum of two numbers  
a = 10 # store 10 into variable a
```

2. Multiline comments

Triple double quotes(“ “ “)

Triple single quotes(“ ‘ ‘)

“ “ “

Web technology encompasses the various tools and techniques used to create, websites.

“ “ “

“ “ “

Web technology encompasses the various tools and techniques used to create websites.

“ “ “

DOCSTRINGS

- **Triple double quotes(" " ") OR Triple single quotes(' ' ') are actually not multiline comments** but they are regular strings with the exception that they can **span multiple lines**.
- So using **Triple double quotes(" " ") OR Triple single quotes(' ' ') are not recommended for comments** in python as they internally occupy memory and would waste time of interpreter.
- If we write strings inside **Triple double quotes(" " ")OR Triple single quotes(' ' ') and if these strings are written as first statements in a module, function or a method, then these strings are called as documentation strings or docstrings**
- **Docstrings are used to create API(application programming interface) documentation file.**
- **An API documentation file is a text file or html file that contains all features of software, language or a product**

Assigning values

```
a=10
```

```
b=15
```

Identifier: names of variables, functions ,objects etc.

In C or java

```
int a;
```

```
a=10
```

In python we need not declare the datatype of variable explicitly

```
a=10
```

Built in datatypes

- Datatypes already available in python are called as built in datatypes.

Built in datatypes are of 5 types

1. NoneType
2. Numeric types
3. Sequences
4. Sets
5. Mappings

Built in datatypes are of 5 types

1. **NoneType**
2. Numeric types
3. Sequences
4. Sets
5. Mappings

The None Type

- **'NoneType'** datatype represents an object that does not contain any value.
- It is used inside a function as a **default value** of the arguments
- When calling the function, if no value is passed the **default value will be taken as none**
- In Boolean **none** value represents **false**.

```
def greet(name=None):  
    if name is None:  
        print("Hello, Stranger!")  
    else:  
        print("Hello, {name}!")
```

```
greet( )      # Output: Hello, Stranger!  
greet("Alice") # Output: Hello, Alice!
```


Built in datatypes are of 5 types

1. **NoneType**
2. **Numeric types**
3. Sequences
4. Sets
5. Mappings

There are 3 subtypes of numeric

- **int**
- **float**
- **complex**

int Type

- Represents an integer number.
- Without any decimal point or fraction part
- Example: 200, -50, 0

a=50

No limit for the size of the datatype

float type

- Floating point numbers
- Number that contains decimal point
- Example: 0.5, -3.5667
- Can also be written in scientific notation using 'e' or 'E' e.g: 2.5E4
- num=55.7

Built in datatypes are of 5 types

1. **NoneType**
2. **Numeric types**
3. Sequences
4. Sets
5. Mappings

There are 3 subtypes of numeric

- int
- float
- **complex**

Complex Datatype

- Written in the form $a+bj$ or $a+bJ$
- 'a' represents real part of the number and 'b' represents the imaginary part.
- 'a' and 'b' may contain integers or floats
- Example: $3+5j$, $-1-5.5J$

$C1=-1-5.5J$

```
# python program to add two complex numbers
c1 = 2.5+2.5j
c2 = 3.0-0.5j
c3 = c1 + c2
print("Sum= ", c3)
```

Output:

```
Sum= (5.5+2j)
```


Built in datatypes are of 5 types

1. **NoneType**
2. **Numeric types**
3. Sequences
4. Sets
5. Mappings

There are 3 subtypes of numeric

- int
- float
- complex

Representing binary, octal and hexadecimal numbers

- **Binary number:** prefix 0b or 0B
0b11110001 , **0B**11100110
- **Hexadecimal number:** prefix 0x or 0X
0xA180 , **0X**11fb91
- **Octal numbers:** Prefix 0o or 0O
0O145 , **0o**773

Built in datatypes are of 5 types

1. **NoneType**
2. **Numeric types**
3. Sequences
4. Sets
5. Mappings

There are 3 subtypes of numeric

- int
- float
- complex

Converting datatypes explicitly

- Python **internally** assumes datatype of the variable.
- If programmer converts one datatype into another datatype on its own it is called as **type conversion**.

`int(x)` is used to convert the number `x` into `int` type.

```
x = 15.56  
int(x) # will display 15
```

`float(x)` is used to convert `x` into `float` type. For example

```
num = 15  
float(num) # will display 15.0
```

Built in datatypes are of 5 types

1. **NoneType**
2. **Numeric types**
3. Sequences
4. Sets
5. Mappings

There are 3 subtypes of numeric

- int
- float
- complex

Converting datatypes explicitly

- Python **internally** assumes **datatype** of the variable.
- If programmer converts one datatype into another datatype on its own it is called as **type conversion**.

`complex(x)` will convert x into a complex number

```
n = 10  
complex(n) # will display (10+0j)
```

`complex(x,y)` will convert x and y into a complex number

```
a = 10  
b = -5  
complex(a, b) # will display (10-5j)
```


A program to convert numbers from octal , binary and hexadecimal systems into decimal number system (Method 1)

```
# python program to convert into decimal number system
n1 = 0017
n2 = 0B1110010
n3 = 0X1c2

n = int(n1)
print('Octal 17 = ', n)
n = int(n2)
print('Binary 1110010 = ', n)
n = int(n3)
print('Hexadecimal 1c2 = ', n)
```

at:

OUTPUT

```
C:\>python convert.py
Octal 17 = 15
Binary 1110010 = 114
Hexadecimal 1c2 = 450
```

Exercise 1:

Write a python program to convert the following numbers in decimal number system.

(a) Octal number 5 (b) Binary number 0010 (c) Hexadecimal number E

```
n1=0o5
n2=0B0010
n3=0xE

n=int(n1)
print('Octal 5=',n)
n=int(n2)
print('Binary 0010=',n)
n=int(n3)
print('Hexadecimal E=',n)
```

OUTPUT:

```
Octal 5= 5
Binary 0010= 2
Hexadecimal E= 14
```


Function `int(string,base)` is useful to convert a string into a decimal/ integer.

A program to convert numbers from octal , binary and hexadecimal systems into decimal number system(Method 2)

```
# python program to convert into decimal number system
s1 = "17"
s2 = "1110010"
s3 = "1c2"

n = int(s1, 8)
print('Octal 17 = ', n)
n = int(s2, 2)
print('Binary 1110010 = ', n)
n = int(s3, 16)
print('Hexadecimal 1c2 = ', n)
```

OUTPUT:

```
Octal 17 = 15
Binary 1110010 = 114
Hexadecimal 1c2 = 450
```


Exercise 2:

Write a python program USING `int(string,base)` to convert the following numbers in decimal number system.

(a) Octal number 5 (b) Binary number 0010 (c) Hexadecimal number E

```
n1="5"
n2="0010"
n3="E"

n=int(n1,8)
print('Octal 5=',n)

n=int(n2,2)
print('Binary 0010=',n)

n=int(n3,16)
print('Hexadecimal E=',n)
```

OUTPUT:

```
Octal 5= 5
Binary 0010= 2
Hexadecimal E= 14
```

```
n1="0o5"
n2="0B0010"
n3="0xE"

n=int(n1,8)
print('Octal 5=',n)

n=int(n2,2)
print('Binary 0010=',n)

n=int(n3,16)
print('Hexadecimal E=',n)
```

bin(): convert number to binary

oct(): convert number to octal

hex (): convert number to hexadecimal

Write a python program to convert 10 to binary, octal and hexadecimal

```
a = 10
b = bin(a)
print(b) # displays 0b1010

b = oct(a)
print(b) # displays 0o12

b = hex(a)
print(b) # displays 0xa
```


Bool datatype

- Bool datatype in python represents boolean values.
- There are only 2 Boolean values **True** or **false**
- **True:1** and **false: 0**
- Conditions will be **evaluated internally** to either true or false.

```
a = 10>5 # here 'a' is treated as bool type variable  
print(a) # displays True
```

```
a = 5>10  
print(a) # displays False
```

```
a = 10  
b = 20  
if(a<b): print("Hello") # displays Hello.
```


Bool datatype

- Bool datatype in python represents boolean values.
- There are only 2 Boolean values **True** or **false**
- **True:1** and **false: 0**
- Conditions will be **evaluated internally** to either true or false.

```
a = 10  
b = 20
```

```
if a < b:  
    print("hello")  
else:  
    print("goodbye")
```

hello

```
a = 20  
b = 10
```

```
if a < b:  
    print("hello")  
else:  
    print("goodbye")
```

goodbye

Built in datatypes are of 5 types

1. **NoneType**
2. **Numeric types**
3. **Sequences(group of elements)**
4. **Sets**
5. **Mappings**

There are 6 subtypes of sequences

- **str**
- **bytes**
- **bytearray**
- **list**
- **Tuple**
- **range**

str Datatype

- String is a group of characters.
- Strings are enclosed in single or double quotes

```
str = "welcome"  
str = 'welcome'
```

We can also write strings inside Triple double quotes(“ “ “) or Triple single quotes(‘ ‘ ‘) to span a group of lines, including spaces.

```
str1 = """This is a book on Python which  
discusses all the topics of Core Python  
in a very lucid manner."""  
  
str2 = '''This is a book on Python which  
discusses all the topics of Core Python  
in a very lucid manner.'''
```


Built in datatypes are of 5 types

1. **NoneType**
2. **Numeric types**
3. **Sequences(group of elements)**
4. **Sets**
5. **Mappings**

There are 6 subtypes of sequences

- **str**
- **bytes**
- **bytearray**
- **list**
- **Tuple**
- **range**

Triple double quotes(“ “ “) or Triple single quotes(‘ ‘ ‘) are useful to **embed a string inside another string**

```
str= “ “ “ This is ‘ Core Python ’ book ” ” ”
```

```
print(str)
```

```
#will display : This is ‘Core Python’ book
```

```
str ‘ ‘ “This is “Core Python” book ’ ’ ’
```

```
print(str)
```

```
#will display : This is “Core Python” book
```

Built in datatypes are of 5 types

1. **NoneType**
2. **Numeric types**
3. **Sequences(group of elements)**
4. **Sets**
5. **Mappings**

There are 6 subtypes of sequences

- **str**
- **bytes**
- **bytearray**
- **list**
- **Tuple**
- **range**

- **Square brackets [] are used to retrieve pieces of a string.**
- **Characters in a string are counted from zero onwards**
- **E.g. str[0] indicates the beginning character**
- *** is repetition character**

str Datatype

- Square brackets [] are used to retrieve pieces of a string.
- Characters in a string are counted from zero onwards
- * is repetition character

```
s = 'welcome to python'  
print((s + ' ') * 2)
```

```
s = 'welcome to Core Python'  
print(s)  
welcome to Core Python
```

w	e	l	c	o	m	e		t	o		c	o	r	e	.	...
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

```
print(s[0]) # display 0th character from s  
W
```

```
print(s[3:7]) # display from 3rd to 6th characters  
come
```

```
print(s[11:]) # display from 11th characters onwards till end  
Core Python
```

```
print(s[-1]) # display first character from the end  
n
```

```
print(s*2)  
welcome to Core Pythonwelcome to Core Python
```


Write a python program to perform the following operations on strings

- a) Take a string as hello world
- b) Display the first character
- c) Display hello
- d) Display world
- e) Print the last character

```
s="hello world"  
print(s[0])  
print(s[0:5])  
print(s[6:])  
print(s[-1])
```

```
h  
hello  
world  
d
```

Write a python program to perform the following operations on strings

- a) Take a string as hello world
- b) Display the first character
- c) Display hello
- d) Display world
- e) Print the last character
- f) **Print string in reverse**

```
s="hello world"  
print(s[0])  
print(s[0:5])  
print(s[6:])  
print(s[-1])  
print(s[::-1])
```

```
h  
hello  
world  
d  
dlrow olleh
```

Built in datatypes are of 5 types

1. **NoneType**
2. **Numeric types**
3. **Sequences(group of elements)**
4. **Sets**
5. **Mappings**

```
elements = [10, 20, 0, 40, 15] # this is a list of byte numbers
x = bytes(elements) # convert the list into bytes array
print(x[0]) # display 0th element, i.e 10
```

There are 6 subtypes of sequences

- **str**
- **bytes**
- **bytearray**
- **list**
- **Tuple**
- **range**

Bytes datatype

- Represents a group of **byte numbers** like an array does
- Byte number is **any positive integer from 0 to 255**
- **Cannot store negative numbers**
- We **cannot modify or edit** any element in the bytes array.

Bytes datatype

Write a python program to create a byte type array, read and display the elements of the array

```
elements=[10,20,0,40,15]    # create a list of byte numbers
x= bytes(elements)           #convert list into bytes type array
for i in x: print(i)         # retrieve elements from x using for loop and display
```

OUTPUT

```
10
20
0
40
15
```

Bytes datatype

Write a python program to create a byte type array, read and display the elements of the array

```
elements = [10, 20, 0, 40, 15]  
x = bytes(elements)  
for i in x:  
    print(i, end=' ')
```

OUTPUT

```
10 20 0 40 15
```

Built in datatypes are of 5 types

1. **NoneType**
2. **Numeric types**
3. **Sequences(group of elements)**
4. **Sets**
5. **Mappings**

There are 6 subtypes of sequences

- **str**
- **bytes**
- **bytearray**
- **list**
- **Tuple**
- **range**

Bytearray datatype

- Similar to bytes datatype
- Difference is that bytes type array cannot be modified but bytearray can be modified.

Bytearray datatype

Write a program to create a bytearray type array with elements as [10,20,0,40,15]. Modify the first two elements and retrieve the elements.

```
# program to understand bytearray type array
# create a list of byte numbers
elements = [10, 20, 0, 40, 15]

# convert the list into bytearray type array
x = bytearray(elements)

# modify the first two elements of x
x[0] = 88
x[1] = 99

# retrieve elements from x using for loop and display
for i in x: print(i)
```

OUTPUT	88
	99
	0
	40
	15

Built in datatypes are of 5 types

1. NoneType
2. Numeric types
3. Sequences(group of elements)
4. Sets
5. Mappings

There are 6 subtypes of sequences

- str
- bytes
- bytearray
- list
- Tuple
- range

List datatype

- List represents a group of elements
- List can store different types of elements but an array can store only one type of elements.
- List can grow dynamically but size of array is fixed.

```
list = [10, -20, 15.5, 'Vijay', "Mary"]
```

- Slicing operation like [0:3] represents elements from 0th to 2nd positions.
- i.e: 10, 20, 15.5

List datatype

```
>>> list = [10, -20, 15.5, 'Vijay', "Mary"]
>>> print(list)
[10, -20, 15.5, 'Vijay', 'Mary']
>>> print(list[0])
10
>>> print(list[1:3])
[-20, 15.5]
>>> print(list[-2])
Vijay
>>> print(list * 2)
[10, -20, 15.5, 'Vijay', 'Mary', 10, -20, 15.5, 'Vijay', 'Mary']
```


List datatype

Write a program to create a list with the following elements[10, 20, “ abc”, “xyz”]. Perform the following operations

- a) Print all the list elements
- b) Print the first element
- c) Print first two elements
- d) Print second and third element
- e) Print last element
- f) Print list two times

```
list=[10,20,"abc","xyz"]  
print(list)  
print(list[0])  
print(list[0:2])  
print(list[1:3])  
print(list[-1])  
print(list *2)
```

OUTPUT

```
[10, 20, 'abc', 'xyz']  
10  
[10, 20]  
[20, 'abc']  
xyz  
[10, 20, 'abc', 'xyz', 10, 20, 'abc', 'xyz']
```

Built in datatypes are of 5 types

1. NoneType
2. Numeric types
3. Sequences(group of elements)
4. Sets
5. Mappings

There are 6 subtypes of sequences

- str
- bytes
- bytearray
- list
- **Tuple**
- range

Tuple datatype

- Tuple contains a group of elements which can be of different types.
- **Elements of a tuple are written inside ()**
- Difference between a list is that the list elements can be modified but **tuple elements cannot be modified. It is a read only list.**

```
tp1 = (10, -20, 15.5, 'Vijay', "Mary")
```

- Individual elements of the tuple can be referenced using square braces as `tp1[0]`, `tp1[2]`....

WE CANNOT MODIFY TUPLE ELEMENTS

Tuple datatype

Write a program to create a tuple with the following elements[10, 20, “ abc”, “xyz”]. Perform the following operations

- a) Print all the tuple elements
- b) Print the first element
- c) Print first two elements
- d) Print second and third element
- e) Print last element
- f) Print tuple two times

```
tp1=(10,20,"abc","xyz")
print(tp1)
print(tp1[0])
print(tp1[0:2])
print(tp1[1:3])
print(tp1[-1])
print(tp1 *2)
```

OUTPUT

```
(10, 20, 'abc', 'xyz')
10
(10, 20)
(20, 'abc')
xyz
(10, 20, 'abc', 'xyz', 10, 20, 'abc', 'xyz')
```


Tuple datatype

Write a program to create a tuple with the following elements[10, 20, “ abc”, “xyz”]. Perform the following operations

- a) Print all the list elements
- b) Print the first element
- c) Print first two elements
- d) Print second and third element
- e) Print last element
- f) Print list two times
- g) **Modify first element to 99**

```
tp1=(10,20,"abc","xyz")
print(tp1)
print(tp1[0])
print(tp1[0:2])
print(tp1[1:3])
print(tp1[-1])
print(tp1 *2)
tp1[0]=99
print(tp1)
```

```
(10, 20, 'abc', 'xyz')
10
(10, 20)
(20, 'abc')
xyz
(10, 20, 'abc', 'xyz', 10, 20, 'abc', 'xyz')
Traceback (most recent call last):
```

```
File C:\Program
Files\Spyder\pkgs\spyder_kernels\py3compat.py:356 in
compat_exec
    exec(code, globals, locals)
```

```
File c:
\users\psurl\onedrive\desktop\imp\python\codes\tuple.py:16
    tp1[0]=99
```

TypeError: 'tuple' object does not support item assignment

Built in datatypes are of 5 types

1. **NoneType**
2. **Numeric types**
3. **Sequences(group of elements)**
4. **Sets**
5. **Mappings**

There are 6 subtypes of sequences

- **str**
- **bytes**
- **bytearray**
- **list**
- **Tuple**
- **range**

range datatype

- range datatype represents a sequence of numbers.
- Example:

```
r = range(10)
```

#will create a range with number starting from 0 to 9

#We can display the numbers using a for loop

```
for i in r : print(i) # will display numbers from 0 to 9
```

```
r=range(30,40,2)
```

```
for i in r: print(i)
```

Will display 30,32,34,36,38 # starting with 30 till 39

We can create a list with a range of numbers from 0 to 9

```
lst = list(range(10))  
print(lst) # will display: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Built in datatypes are of 5 types

1. **NoneType**
2. **Numeric types**
3. **Sequences**
4. **Sets**
5. **Mappings**

There are two subtypes in sets

- **set datatype**
- **frozenset datatype**

Set datatype

To create a **set** you should enter the elements separated by commas inside **curly braces { }**

```
s = {10, 20, 30, 20, 50}
print(s) # may display {50, 10, 20, 30}
```

- Set is not maintaining order
- Set does not store duplicate elements

```
ch = set("Hello")
print(ch) # may display {'H', 'e', 'l', 'o'}
```

- We can convert a list into a set using the set function

```
lst = [1, 2, 5, 4, 3]
s = set(lst)
print(s) # may display {1, 2, 3, 4, 5}
```


Built in datatypes are of 5 types

1. NoneType
2. Numeric types
3. Sequences
4. Sets
5. Mappings

There are two subtypes in set

- set datatype
- frozenset datatype

Set datatype

Since set is unordered we cannot retrieve the elements using indexing operations

```
print(s[0])  
print(s[0:2])
```

CANNOT BE USED

- `update()` method is used to **add** elements to the set

```
s.update([50,60])  
print(s) # may display {1, 2, 3, 4, 5, 50, 60}
```

- `remove()` method is used to **remove** any particular element from the set.

```
s.remove(50)  
print(s) # may display {1, 2, 3, 4, 5, 60}
```

Built in datatypes are of 5 types

1. NoneType
2. Numeric types
3. Sequences
4. Sets
5. Mappings

There are two subtypes in sets

- set datatype
- **frozenset datatype**

frozenset datatype

- Similar to set datatype
- Difference is that elements in set datatype can be modified but **elements in frozenset datatype cannot be modified.**
- Example:

```
s={10, 20, 30,50}  
fs= frozenset(s) # create frozenset  
print(fs)
```

Output: *frozenset({10, 20, 50, 30})*

Another way of creating a frozen set is by passing a string (group of characters) to the frozen set.

```
fs=frozenset("abcdefg")  
print(fs) # may display frozenset({'e' ,'g' ,'f', 'd', 'a', 'c', 'b'})
```

Update() and remove() methods will not work on frozen sets as they cannot be modified

Built in datatypes are of 5 types

1. NoneType
2. Numeric types
3. Sequences
4. Sets
5. Mappings

Mapping types

- Map represents a group of elements in the form of key/value pairs
- **dict datatype is an example for a map:** dict represents a dictionary that contains pairs of elements such that first element represents the key and the next one becomes its value.
- **COLON (:) to separate key and value.**
- **Pair separated by comma.**
- **All elements inside curly brackets.**
- **Example: roll no and names of students.**
`d={10 : 'Rohan' , 11: 'Krishna' ,12: 'Ram'}`

OR

An empty dictionary can be created

```
d= { }
```

Later we can store key and values into d as

```
d[10] : ' Rohan'
```

```
d[11] : ' Krishna'
```

```
print(d)
```

Will display

```
{10 : 'Rohan' , 11: 'Krishna'}
```


Built in datatypes are of 5 types

1. NoneType
2. Numeric types
3. Sequences
4. Sets
5. Mappings

Mapping types

Performing operations:

d[key] : To retrieve values upon giving the keys

keys() : to retrieve only keys

values() : to retrieve only values

d[key]=newvalue : to update value

del d[key] : will delete key along with its values

Mapping types

```
>>> d = {10: 'Kamal', 11: 'Pranav', 12: 'Hasini', 13: 'Anup', 14: 'Reethu'}
>>> print(d)
{10: 'Kamal', 11: 'Pranav', 12: 'Hasini', 13: 'Anup', 14: 'Reethu'}
>>> print(d[11])
Pranav
>>> print(d.keys())
dict_keys([10, 11, 12, 13, 14])
>>> print(d.values())
dict_values(['Kamal', 'Pranav', 'Hasini', 'Anup', 'Reethu'])
>>> d[10] = 'Hareesh'
>>> print(d)
{10: 'Hareesh', 11: 'Pranav', 12: 'Hasini', 13: 'Anup', 14: 'Reethu'}
>>> del d[11]
>>> print(d)
{10: 'Hareesh', 12: 'Hasini', 13: 'Anup', 14: 'Reethu'}
```

Literals in python

Literal is a constant value that is stored into a variable in a program.

Example:

a =10

Following are different types of literals in python

- **Numeric literals**
- **Boolean literals**
- **String literals**

- **Numeric literals**

Literals in python

Types of literals in python

- **Numeric literals**
- Boolean literals
- String literals

Examples	Literal name
450, -15	Integer literal
3.14286, -10.6, 1.25E4	Float literal
0x5A1C	Hexadecimal literal
0557	Octal literal
0B110101	Binary literal
18+3j	Complex literal

Literals in python

Types of literals in python

- Numeric literals
- **Boolean literals**
- String literals

Boolean literals

- Boolean literals are the **true** or **false** values stored into a bool type variable

Literals in python

Types of literals in python

- Numeric literals
- Boolean literals
- **String literals**

String literals

- A group of characters is called a string literal
- String literals are enclosed in
 - ❑ single quotes(')
 - ❑ Double quotes(")
 - ❑ Triple quotes("'' or ''''')

In python there is no difference between single quoted strings and double quoted strings

```
s1 = 'This is first Indian book'  
s2 = "Core Python"
```


Literals in python

Types of literals in python

- Numeric literals
- Boolean literals
- String literals

String literals

- When a string literal extends beyond a single line we should use triple quotes

```
s1 = '''This is first Indian book on  
Core Python exploring all important  
and useful features'''  
  
s2 = """This is first Indian book on  
Core Python exploring all important  
and useful features"""
```

If you want to use double quotes and a string spans more than one line adding Backslash(\) will join next string to it.

```
str = "This is first line and \n  
this will be continued with the first line"  
print(str)  
This is first line and this will be continued with the first line
```

String literals

```
str = "This is \nPython"  
print(str)  
This is  
Python
```

Escape character	Meaning
\	New line continuation
\\	Display a single \
\'	Display a single quote
\"	Display a double quote
\b	backspace
\r	Enter
\t	Horizontal tab space
\v	Vertical tab
\n	New line

Determining the datatype of a variable

- To know the **datatype** of a variable or object we can use the `type()` function.

```
a= 15
print(type (a))

a= 15.5
print(type (a))

s= "hello"
print(type (s))

print(type("hello"))

lst=[1,2,3]
print(type(lst))

t=(1,2,3,4)
print(type(t))

s= {1,2,3,4}
print( type(s))
```

OUTPUT:

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'str'>
<class 'list'>
<class 'tuple'>
<class 'set'>
```

Every datatype, function, method, class, lists, sets etc are all objects in python.

Determining the datatype of a variable

- Determining type for characters

Python does not have a char datatype to represent individual characters.

It has str datatype which represents strings.

```
ch = 'A'  
print(type(ch))  
<class 'str'>
```

```
str = 'Bharat' # will display B  
print(str[0])
```

```
for i in str: print(i)
```

it will display the characters as:

```
B  
h  
a  
r  
a  
t
```

```
str = 'Bharat'  
print(str[0])
```

```
str[0].isupper() #will display true
```

```
str[1].isupper() # will display false
```

User defined datatypes

Datatypes which are created by programmers are called user defined datatypes.

Example:

Array, class etc are user defined datatypes

Mod 3

Constants in python

- Constant is similar to a variable but its **value cannot be changed or modified**.
- Once defined a constant cannot allow changing its value.
- A **programmer can indicate a variable as constant by writing its name in capital letters**

Identifiers and reserved words

Identifiers

- Is a **name** given to a variable or function or class etc.
- **Can include letters, numbers and underscore(_)**
- Always start with a **non numeric character**.
- Special symbols such as **?, # \$, %, and @** are not allowed.
- **Examples:** salary, abc11, abc_x etc.
- Python is **case sensitive language**.
- **Num and num** are treated as different names.

Identifiers and reserved words

salary = 15000.75

This symbol stores right side literal into left side variable. Thus it is performing an operation. Hence it is called 'operator'.

This is a variable.
Its name is 'salary'. Hence this name 'salary' is 'identifier'

This is the float number stored into variable. Hence it is called 'floating point literal'.

Reserved words

Reserved words should not be used as identifiers.

Reserved words available in Python:

and	del	from	nonlocal	try
as	elif	global	not	while
assert	else	if	or	with
break	except	import	pass	yield
class	exec	in	print	False
continue	finally	is	raise	True
def	for	lambda	return	

Naming Conventions in Python

- Rules related to writing names of packages, modules, classes, variables etc are called naming conventions.
 - The following naming conventions should be followed.
1. **Packages:** Packages names should be written in all lower case letters. When multiple words are used for a name, we should separate them using an underscore(_).
 2. **Modules:** Module names should be written in all lower case letters. When multiple words are used for a name, we should separate them using an underscore(_).
 3. **Classes:** Each word of a class name should start with a capital letter.(rule applicable to the class created by us).Pythons build in class names use all lowercase letters.(eg: int, str, list etc.)
 4. **Global variables or module level variables:** should be written in all lower case letters. When multiple words are used for a name, we should separate them using an underscore(_).
 5. **Instance variables:** should be written in all lower case letters. When multiple words are used for a name, we should separate them using an underscore(_).
 6. **Functions:** should be written in all lower case letters. When multiple words are used for a name, we should separate them using an underscore(_).
 7. **Method arguments:** In case of instance methods, the first argument name should be 'self'. In case of class methods, first argument must be 'cls'.
 8. **Constants:** Constants names should be written in all capital letters. If there are several words then each word must be separated by an underscore(_).

SUMMARY OF DATATYPES

Built in datatypes are of 5 types

1. NoneType
2. Numeric types
3. Sequences
4. Sets
5. Mappings

1. None Type
2. Numeric types

- **int**
- **float**
- **complex**
- **Conversions(octal ,bin,hex)**
- **Bool type**

3 . sequences

- **str**
- **bytes** : numbers from 0 to 255, cannot modify elements
- **bytearray** : numbers from 0 to 255, can modify elements
- **list** : square brackets[], contain any type of elements, can modify elements
- **tuple** : round brackets(), contain any type of elements, cannot modify elements
- **range** : creates a range of numbers

SUMMARY OF DATATYPES

Built in datatypes are of 5 types

1. NoneType
2. Numeric types
3. Sequences
4. Sets
5. Mappings

4. Sets

- **set datatype**: curly brackets { }, elements unordered, no indexing operations, can modify elements.
- **frozenset datatype**: curly brackets { }, elements unordered, cannot modify elements.

5. Mappings: key value pairs, colon operator(:) to separate key and value, curly brackets { }

THANK YOU😊

by PRACHI SURLAKAR